Student Profiling System

A role-based Java console application for student records, academics and activities.

03 user roles	**05** database tables	**02** activity categories

HOW THE SYSTEM IS ORGANIZED

USER EXPERIENCE
Student / Teacher / Administrator menus

↓

APPLICATION LOGIC
StudentProfilingSystem ? role menus ? PermissionGuard

↓

DATA ACCESS
JDBC DAOs ? parameterized SQL ? transaction for student creation

↓

PERSISTENCE
MySQL schema ? users, student profiles, academic and activity records

PROJECT PURPOSE

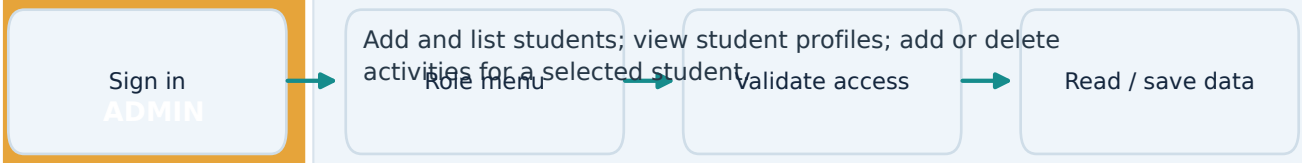
Centralize student profiles and related academic and co-curricular records, while giving each user role a focused workflow. Data persists in MySQL; Java code is organized around domain models, a console interface, and DAO-based database access.

Roles & capabilities

Each role enters a separate menu after authentication.

STUDENT	View own profile and academic records; view, add and delete own curriculum and non-curriculum activities; update own email and password.
TEACHER	Look up student profiles; add academic marks and attendance; view curriculum and non-curriculum activities.
ADMIN	Add and list students; view student profiles; add or delete activities for a selected student.

TYPICAL REQUEST FLOW



Ownership-aware deletion

Student activity deletes match both the activity ID and the logged-in student ID in SQL.

Parameterized database access

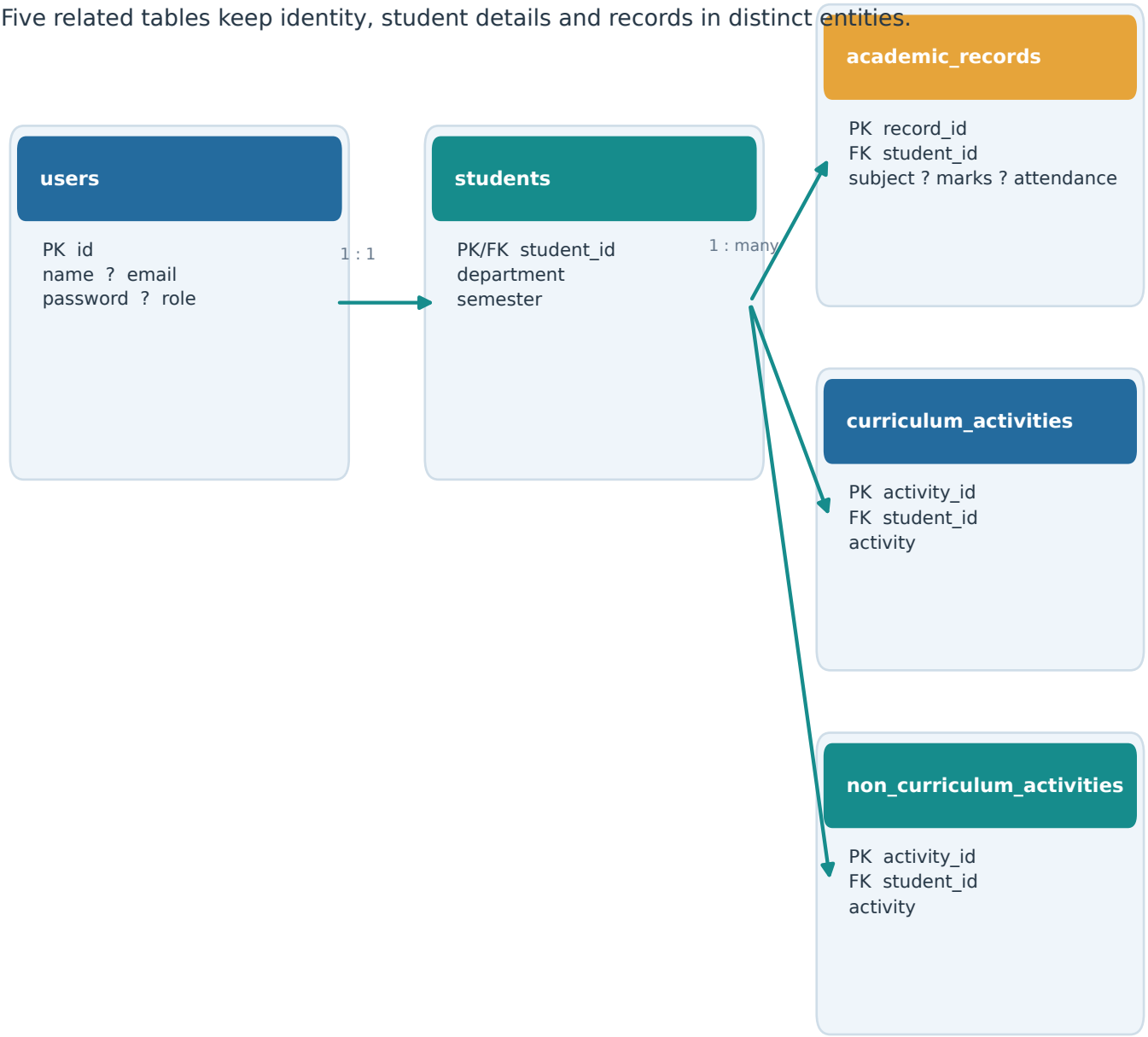
DAO operations bind user input through PreparedStatement parameters rather than SQL concatenation.

SCOPE NOTE

Role checks and ownership safeguards are implemented for the listed console flows. Password hashing and stronger account lifecycle management remain future work.

Data model

Five related tables keep identity, student details and records in distinct entities.



RELATIONAL RULES

- A student row belongs to a user row; student-owned records reference students.student_id.
- Foreign keys cascade deletion of dependent academic and activity records.
- Database checks constrain roles, semester (1?12), marks (0?100) and attendance (0?100).

Schema definition: schema.sql

DELIVERY GUIDE / OPERATIONS

Setup & next steps

Requirements

JDK 9+ (Java 8 API target) ? Maven 3.6+ ? MySQL 8.0.16+

01

Create database

Run schema.sql using a MySQL account allowed to create the database and tables.

02

Bootstrap admin

Insert the first ADMIN row into users; use a unique ID and email.

03

Configure connection

Set DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD in your shell.

04

Build and launch

Run the Maven commands below from the project folder.

POWERSHELL / BUILD & RUN

```
mvn clean compile
mvn exec:java
```

LIMITATIONS & PRIORITIES

Educational prototype: passwords are currently plain text. Before real deployment, add salted password hashing, stronger input and account validation, automated MySQL integration tests, and account-management workflows.